# Use Case Diagram

A **Use Case Diagram** in Unified Modeling Language (**UML**)

**Definition**: A visual representation of interactions between users (actors) and a system.
•**Purpose**: Captures functional requirements and system behavior.

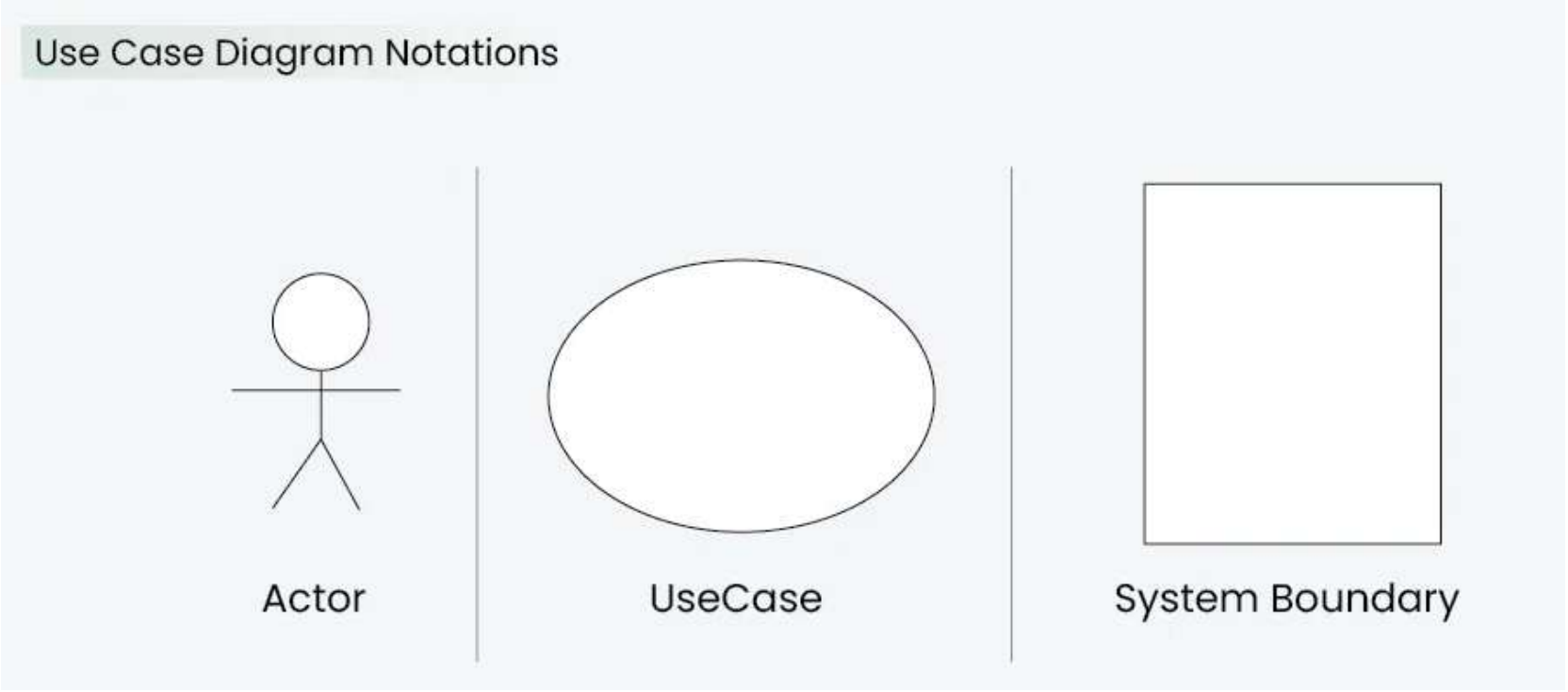
•**Components**:
- •**Actors**: Represent users or external systems interacting with the system.
- •**Use Cases**: Specific functionalities or processes within the system.
- •**Relationships**: Show interactions between actors and use cases.

•**Benefits**:
- •Provides a high-level overview of system functionality.
- •Helps stakeholders, developers, and analysts understand system behavior.
- •Defines system scope and requirements clearly.
- •Establishes relationships between different processes.

•**Usage**: Essential in requirement gathering and system design phases.

# What is a Use Case Diagram in UML?

•A Use Case Diagram is a type of Unified Modeling Language (UML) diagram that represents the **interaction between actors (users or external systems) and a system** under consideration to accomplish specific goals.
•It provides a high-level view of the system's functionality by illustrating the various ways users can interact with it.



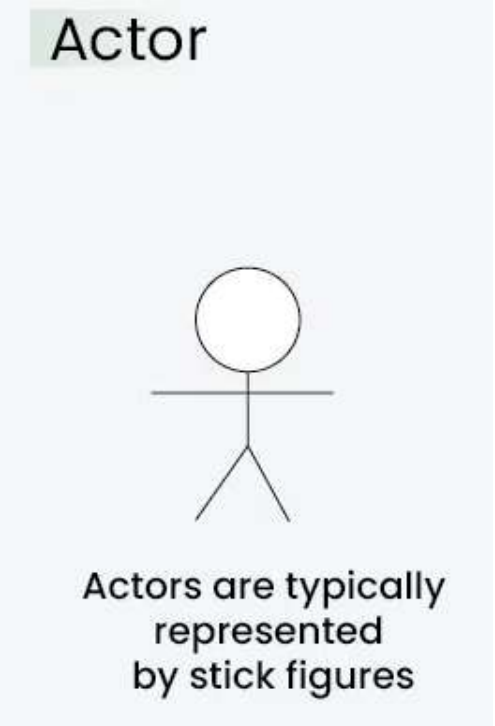
# When to apply Use Case Diagram?

- Use case diagrams are useful in several situations. Here’s when you should consider using them:
- **Requirement Gathering:** Helps visualize how different users interact with the system.
- **Stakeholder Communication:** Provides a clear, simple way to convey system functionality, especially to non-technical stakeholders.
- **System Design:** Outlines user interactions and plans features to ensure alignment with user needs.
- **Scope Definition:** Clarifies system boundaries by distinguishing between internal and external elements.

# Use Case Diagram Notations

- UML notations visually represent system designs, architectures, and behaviors, ensuring clear and consistent communication among stakeholders.

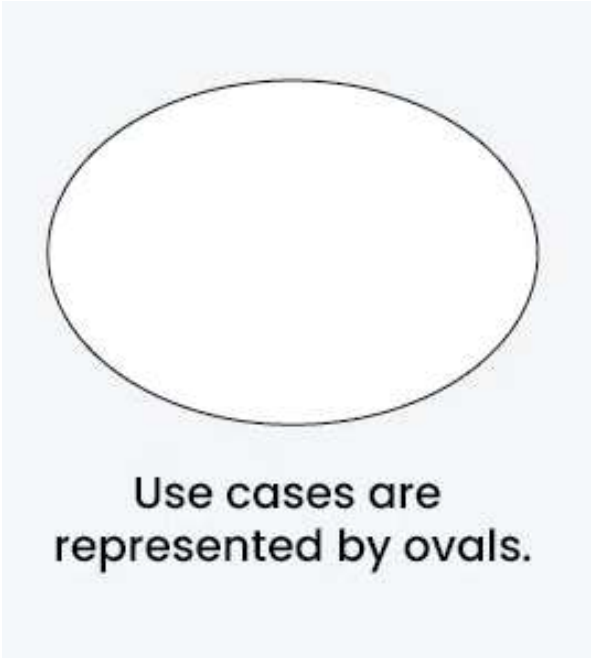
1. **Actors:**
    - Actors are external entities that interact with the system.
    - These can include users, other systems, or hardware devices. In the context of a Use Case Diagram, actors initiate use cases and receive the outcomes.
    - Proper identification and understanding of actors are crucial for accurately modeling system behavior.



# Use Case Diagram Notations

## 2. Use Cases

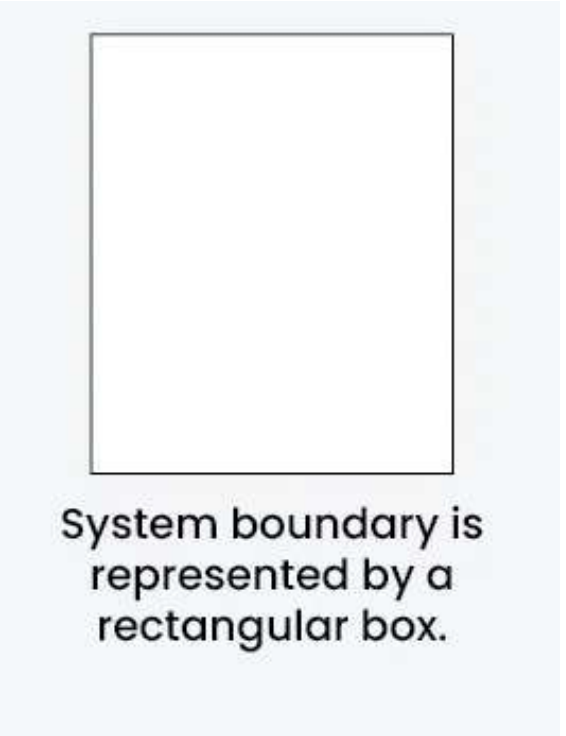
- Use cases are like scenes in the play.
- They represent specific things your system can do. In the online shopping system, examples of use cases could be "Place Order," "Track Delivery," or "Update Product Information".
- Use cases are represented by ovals.



# Use Case Diagram Notations

### 3. System Boundary

- The system boundary is a visual representation of the scope or limits of the system you are modeling.
- It defines what is inside the system and what is outside. The boundary helps to establish a clear distinction between the elements that are part of the system and those that are external to it.
- The system boundary is typically represented by a rectangular box that surrounds all the use cases of the system.
- The purpose of system boundary is to clearly outlines the boundaries of the system, indicating which components are internal to the system and which are external actors or entities interacting with the system.



# Use Case Diagram Relationships

- Relationships show how actors interact with use cases, providing a clear view of system functionality.
- Key types include association, include and extend.

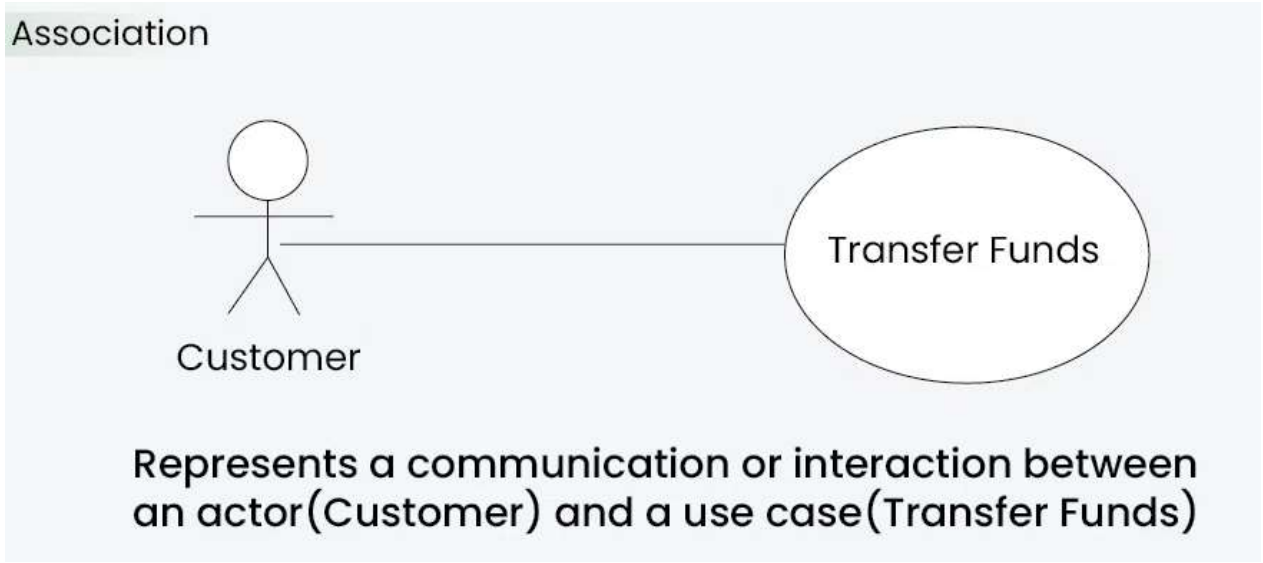
**1. Association Relationship**

- The Association Relationship represents a communication or interaction between an actor and a use case. It is depicted by a line connecting the actor to the use case. This relationship signifies that the actor is involved in the functionality described by the use case.

# Use Case Diagram Relationships

**Example: Online Banking System**

- Actor: Customer
- Use Case: Transfer Funds
- Association: A line connecting the "Customer" actor to the "Transfer Funds" use case, indicating the customer's involvement in the funds transfer process.

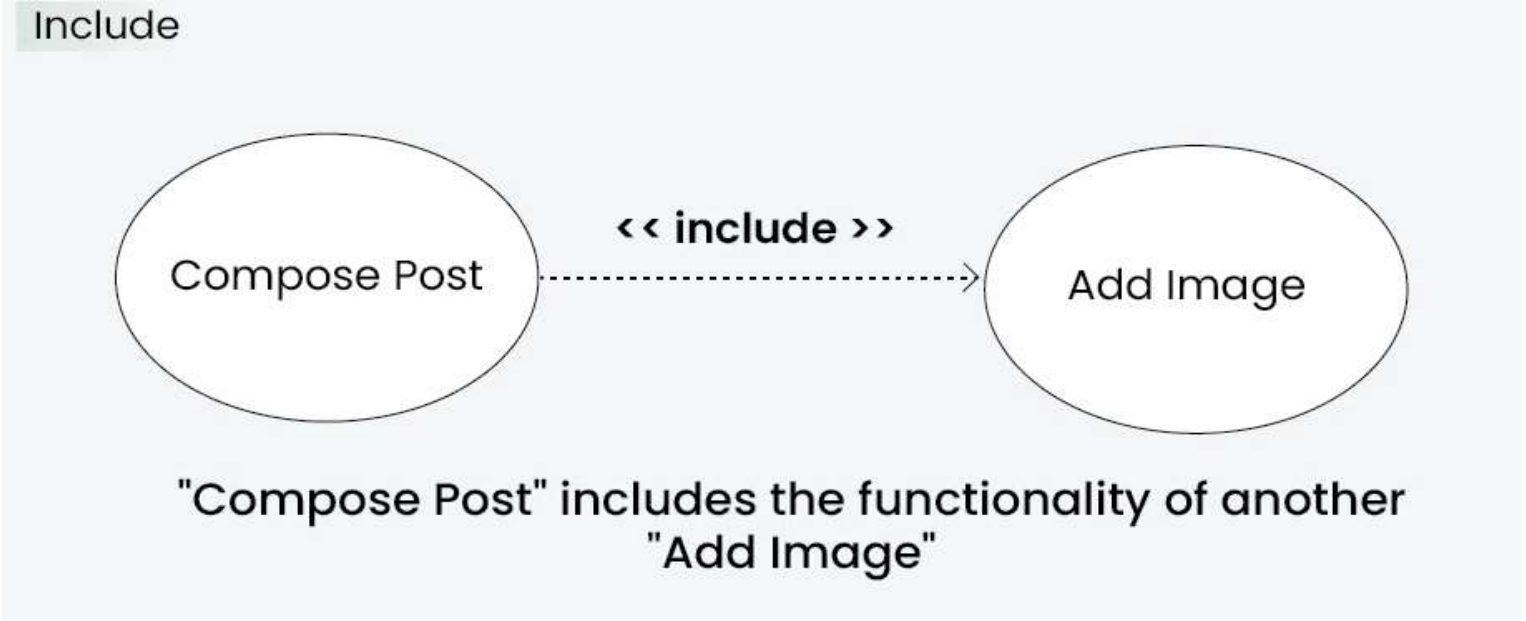


## 2. Include Relationship

- The Include Relationship indicates that a use case includes the functionality of another use case.
- It is denoted by a dashed arrow pointing from the including use case to the included use case.
- This relationship promotes modular and reusable design.

**Example: Social Media Posting**

- Use Cases: Compose Post, Add Image
- Include Relationship: The "Compose Post" use case includes the functionality of "Add Image." Therefore, composing a post includes the action of adding an image.

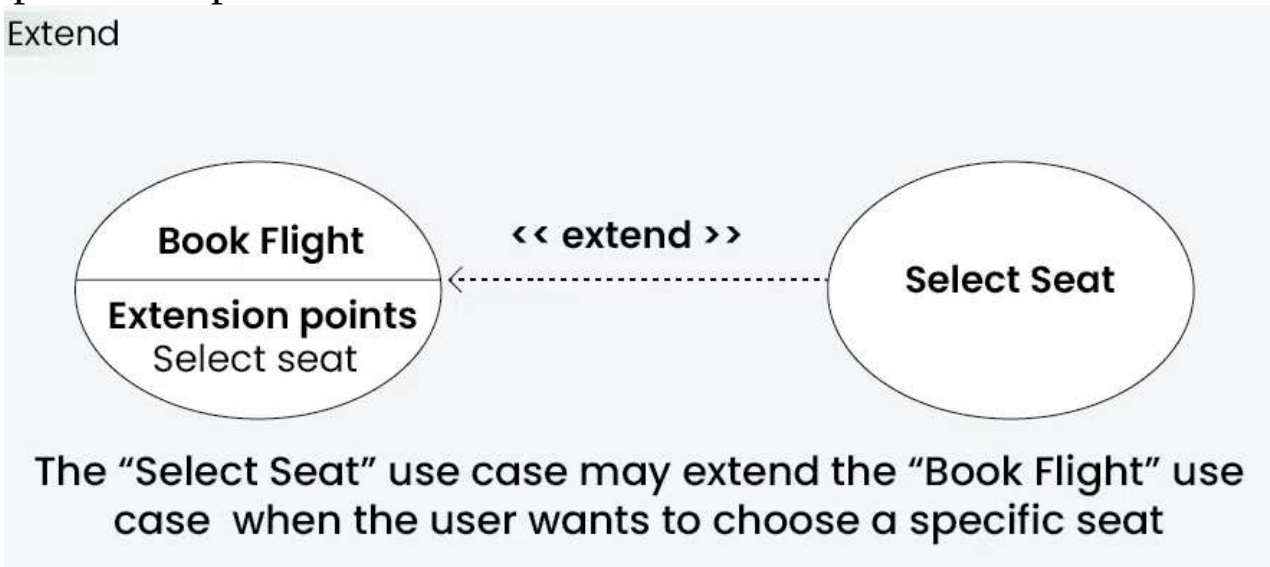


### 3. Extend Relationship

- The Extend Relationship illustrates that a use case can be extended by another use case under specific conditions.
- It is represented by a dashed arrow with the keyword "extend."
- This relationship is useful for handling optional or exceptional behavior.

**Example: Flight Booking System**

- Use Cases: Book Flight, Select Seat
- Extend Relationship: The "Select Seat" use case may extend the "Book Flight" use case when the user wants to choose a specific seat, but it is an optional step.

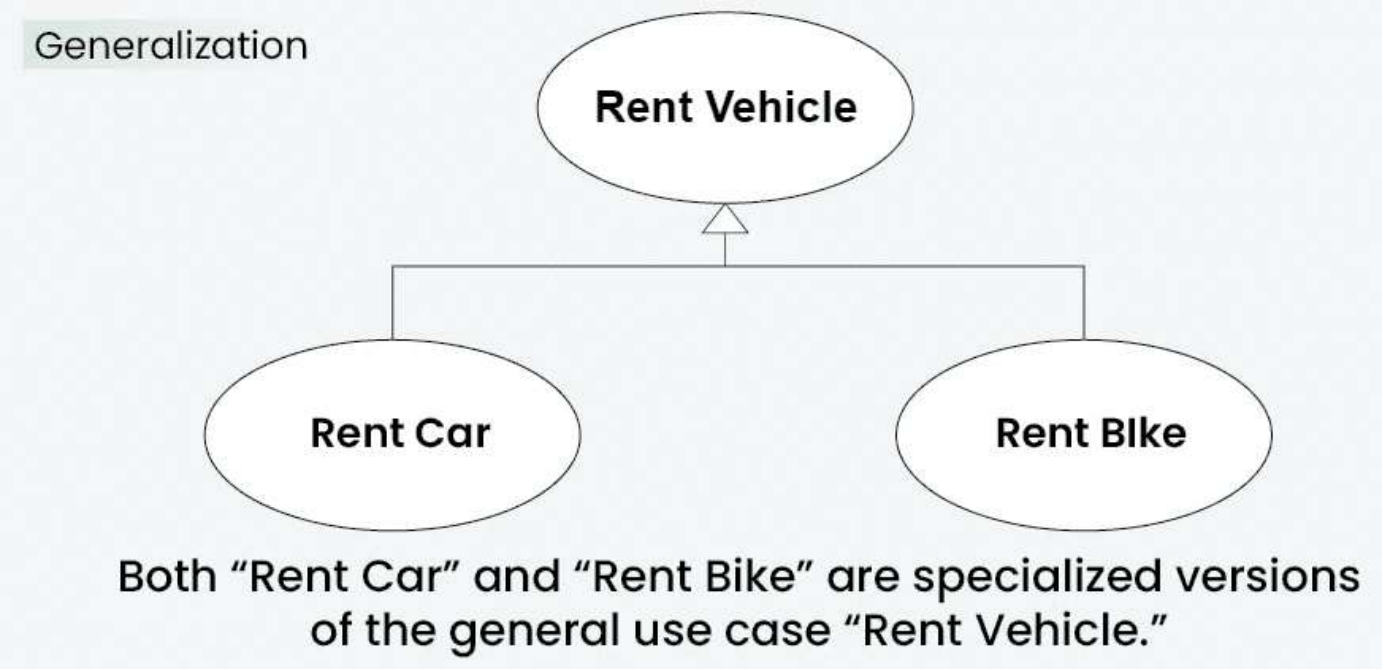


### 4. Generalization Relationship

- The Generalization Relationship establishes an "is-a" connection between two use cases, indicating that one use case is a specialized version of another.
- It is represented by an arrow pointing from the specialized use case to the general use case.

**Example: Vehicle Rental System**

- Use Cases: Rent Car, Rent Bike
- Generalization Relationship: Both "Rent Car" and "Rent Bike" are specialized versions of the general use case "Rent Vehicle."

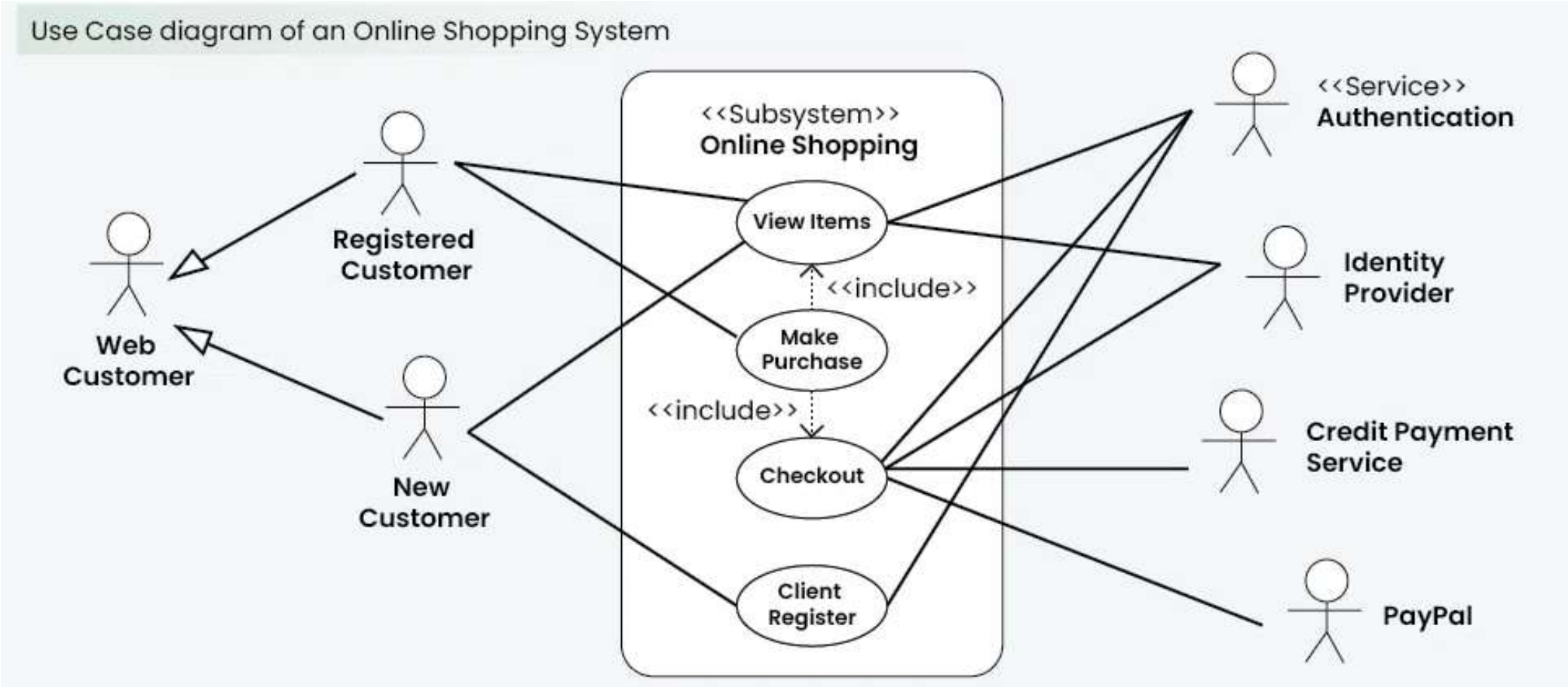


**How to draw a Use Case diagram in UML?**

- Below are the **main steps** to draw use case diagram in UML:
- **Step 1: Identify Actors:** Determine who or what interacts with the system. These are your actors. They can be users, other systems, or external entities.
- **Step 2: Identify Use Cases:** Identify the main functionalities or actions the system must perform. These are your use cases. Each use case should represent a specific piece of functionality.
- **Step 3: Connect Actors and Use Cases:** Draw lines (associations) between actors and the use cases they are involved in. This represents the interactions between actors and the system.
- **Step 4: Add System Boundary:** Draw a box around the actors and use cases to represent the system boundary. This defines the scope of your system.
- **Step 5: Define Relationships:** If certain use cases are related or if one use case is an extension of another, you can indicate these relationships with appropriate notations.
- **Step 6: Review and Refine:** Step back and review your diagram. Ensure that it accurately represents the interactions and relationships in your system. Refine as needed.
- **Step 7: Validate:** Share your use case diagram with stakeholders and gather feedback. Ensure that it aligns with their understanding of the system's functionality.

# Use Case Diagram example(Online Shopping System)

- **Actors:**
  - Customer
  - Admin
- **Use Cases:**
  - Browse Products
  - Add to Cart
  - Checkout
  - Manage Inventory (Admin)
- **Relations:**
  - The Customer can browse products, add to the cart, and complete the checkout.
  - The Admin can manage the inventory.


Use Case diagram of an Online Shopping System
<<Subsystem>>
Online Shopping
View Items
<<include>>
Make Purchase
<<include>>
Checkout
Client Register
Registered Customer
Web Customer
New Customer
<<Service>>
Authentication
Identity Provider
Credit Payment Service
PayPal

# What are Common Mistakes while making Use Case Diagram?

- Avoiding common mistakes ensures the accuracy and effectiveness of the Use Case Diagram. Here are key points for each mistake:
- Adding too much detail can confuse people.
- Unclear connections lead to misunderstandings about system interactions.
- Different names for the same elements create confusion.
- Incorrectly using generalization can misrepresent relationships.
- Failing to define the system’s limits makes its scope unclear.
- Treating the diagram as static can make it outdated and inaccurate.

- **Conclusion**
- In conclusion, a Use Case Diagram in UML serves as a powerful tool for capturing and visualizing the functional requirements and interactions within a system.
- By representing actors, use cases, and their relationships in a clear and concise manner, this diagram provides a high-level overview of the system's behavior.